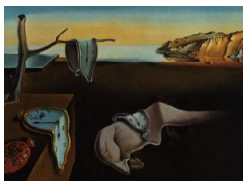

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

treecompareR:

Paul Kruse
US Environmental Protection Agency

Caroline Ring
US Environmental Protection Agency

Abstract

This paper presents the **treecompareR** package for R, which provides tools for reproducible visualizations of data through the use of taxonomies. The package builds on developments from **ggplot2** and **ggtree** to provide visualizations tailored for use with taxonomic classification data. It also provides a pipeline for gathering chemical classification data using **classyfireR**. Additionally, it provides tools that leverage developments in network analysis to compare data sets. While designed specifically for chemical classification objectives using ClassyFire(`http://classyfire.wishartlab.com/`), **treecompareR** provides tools for use with more general taxonomies.

Keywords: JSS, style guide, comma-separated, not capitalized, R.

Classification of objects has been a source of much development of organizational structures across various fields of science. Organizing objects into a coherent structure helps one to understand how these objects relate to each other and can lead to the discovery of connections between various sets of objects. How one organizes the objects may depend on underlying attributes that the objects possess.

Ontologies provide a framework for comparing objects through a variety of different relation types. These relation types create a network structure that can be studied using graph theory techniques. One of the more actively developed ontologies in recent science is the Gene Ontology, which aims to develop a computation model of biological systems. The ontology considers the three domains defined as molecular function, cellular component, and biological process with relationships between terms within each of these domains and between the three domains as well. Using these domains, the ontology describes genes and how they relate to various areas of interest in contemporary biological research.

Within the universe of ontologies sits the set of organizing structures known as taxonomies. Restricting the attention to taxonomies, one can focus specifically on the "is a" relation, e.g. "a dog (canis familiaris) is a canis" and study how objects relate to one another under this specific relationship scheme. One well known example of this organizational structure in use is the Linnaean system for classifying life, sometimes referred to as the "tree of life". As a

taxonomy, the organizational structure is given by a tree, which consequently lends itself to analysis using tree-based techniques from graph theory.

In 2016, the ClassyFire (http://classyfire.wishartlab.com/) tool was introduced, in the work of Djoumbou Feunang, Eisner, Knox, Chepelev, Hastings, Owen, Fahy, Steinbeck, Subramanian, Bolton *et al.* (2016). This work presented a chemical ontology, ChemOnt, consisting of 4825 classification labels. These labels are organized in a taxonomy structure with pairs of distinct labels either satisfying a simple "is a" relation or not. The ClassyFire tool takes in chemical identifiers such as InChIKey or SMILES strings and provides a classification of the chemical within the taxonomy. In this manner, one can examine a data set consisting of chemical data from the viewpoint of where the chemicals lie within the ChemOnt taxonomy structure.

In the study of cheminformatics, machine-learning models such as Quantitative structure-activity relationship (QSARs) are constructed to link physical and structural properties with biological interactions. For these models to produce accurate and reliable predictions, an analysis of the training data and targeted use data is essential. This consideration of the domain of applicability of the model is important for determining when the trained model's predictions can be considered reliable given an input chemical. In many cases, this analysis can be carried out using model descriptors, for example by using structural descriptors of the molecules used for training the model. However, while descriptors may accurately describe the domain of applicability, they often can be hard to interpret and they do not necessarily lend themselves that well to visualization.

Understanding and overcoming these issues in one's ability to interpret the data are especially important when comparing pairs of data sets, such as training and test sets or training and prediction sets. While it may be feasible to examine how each individual data set is characterized by model descriptors, how they overlap requires additional consideration. Is the overlap broad or does it focus on specific areas? Are there gaps between the training data and prediction data that may reduce the confidence users have in model predictions? Answering these and other questions addresses concerns of model applicability to data sets in general and how the model can be improved regarding gaps in training data compared to prediction or test data.

One approach to addressing these naturally arising concerns is through visualizations that can increase the user's ability to interpret the data sets. Since the ClassyFire ChemOnt ontology is organized in a taxonomy structure, one can leverage the tree structure not only for quantitative analysis but also through the use of tree visualizations. These visualizations can reveal the extent to which there are overlaps within the data and the degree of such overlaps. The diagrams also show very clearly when there are gaps between the data sets and broad coverage, or lack thereof, within the chemical universe as it is represented by ChemOnt.

However, such visualizations are limited in terms of what is currently available. One may use the **ggplot2** R package to visualize statistics based on the taxonomic information corresponding to classifications of a data set Wickham (2016) . But **ggplot2** is a very broad package and not designed for handling chemical classification data in particular. Since the ChemOnt taxonomy is given by a tree structure, one can use tree visualization packages such as **ggtree** to visualize this structure Yu, Smith, Zhu, Guan, and Lam (2017). However, the **ggtree** package was designed with phylogenetic trees in mind, which are generally presented as rooted binary trees (possibly with additional data). The ChemOnt ontology is neither a binary tree nor does it carry additional explicit data, so **ggtree** is again a very broad approach to

this. The package **treecompareR** leverages several R packages designed for tree manipulation and exploration, including **phytools** Revell (2024), **phangorn** Schliep (2011); Schliep, Potts, Morrison, and Grimm (2017), **Ape** Paradis and Schliep (2019), **Treeio** Wang, Lam, Xu, Dai, Zhou, Feng, Guo, Dunn, Jones, Bradley, Zhu, Guan, Jiang, and Yu (2020). It also combines tree visualizations with heatmaps, in particular using the package **ComplexHeatmap** Gu, Eils, and Schlesner (2016).

We present the package **treecompareR** as a solution to the specific needs of creating a pipeline to classify chemicals in a data set, visualize the chemical space they represent, and compare the data sets from the point of view of chemical space. We introduce the package through a case study analyzing two specific data sets. We characterize two data sets, named `BIOSOLIDS` and `USGSWATER`, that were collected from the CompTox Chemical Dashboard Williams, Grulke, Edwards, McEachran, Mansouri, Baker, Patlewicz, Shah, Wambaugh, Judson *et al.* (2017). The `BIOSOLIDS` data describe chemicals detected in biosolids, as reported by three national sewage sludge surveys and eight biennial reviews, while the `USGSWATER` data describe chemicals found in water by the USGS.

We are interested in studying these two data sets together for a variety of reasons. The presence of chemicals detected in water is important information to consider when determining risk assessment for human and ecological health. Not only does this indicate what chemicals human activity contributes to water, but also what to expect to be present in water that is used in commercial, industrial, and residential settings. Chemicals detected in biosolids indicate both the chemicals that are already present in water in before being used, and the chemicals that human activity contributes to water and persists through the treatment process of waste water. By examining these two sets, we can get a better idea of which chemicals are introduced directly through human activity and use, and which chemicals are already in the background water as well as those that may be successfully removed from the water and captured within the biosolid.

1. ClassyFire

In this paper, we use the `chemical_list_BIOSOLIDS_2022_05_10` and `chemical_list_USGSWATER_2022_05_` data sets, which are available for download from the US Environmental Protection Agency's Comptox Chemicals Dashboard `https://comptox.epa.gov/dashboard/chemical-lists`. Note, the names of these data sets as listed on the CompTox Dashboard Chemical List are `BIOSOLIDS2021` and `USGSWATER`, respectively.

These data sets include a list of chemicals, relevant identifiers such as `DTXSID`, `InChIKey`, `SMILES`, physico-chemical property data such as `MOLECULAR_FORMULA`, `AVERAGE_MASS`, `MONOISOTOPIC_MASS`, and publication and quality data such as `PUBCHEM_DATA_SOURCES` and `QC_Level`.

Before we start comparing these data sets, we must provide each chemical in each data set classification data from ClassyFire. To do this, we use the classification pipeline in **treecompareR**, construct a tree that encodes the taxonomy used in this classification scheme, and use this tree for data analysis and to create visualizations.

To achieve these aims, we use the ClassyFire tool found at `http://classyfire.wishartlab.com/`. ClassyFire uses the ChemOnt ontology, consisting of 4825 labels over 11 levels of classification, to classify chemicals. The ontology is given a taxonomic structure, that of a tree, with each label given a unique parent label and potentially having several chil-

dren labels. This relationship data between labels can be found at the ChemOnt Taxnodes page http://classyfire.wishartlab.com/tax_nodes.json in JSON format. To construct a tree that encodes this structure, we must first download this data or use the file `chemont_parent_child.rda` from **treecompareR** with column names `Name`, `ID`, `Parent_ID`. With this data in hand, we construct the tree as a `phylo` object as follows.

```
R> chemont_taxonomy <- generate_taxonomy_tree(tax_nodes = chemont_df)
R> str(chemont_taxonomy[[1]], width = 60)

 num [1:4824, 1:2] 3613 3614 3615 3615 3615 ...

R> str(chemont_taxonomy[[2]], width = 60)

 int 1213

R> str(chemont_taxonomy, width = 60)

List of 5
 $ edge       : num [1:4824, 1:2] 3613 3614 3615 3615 3615 ...
 $ Nnode      : int 1213
 $ tip.label  : chr [1:3612] "Thiepanes" "Benzoxazolines" "Polychlorinated dibenzofurans"
 $ edge.length: num [1:4824] 9.09 11.36 79.55 79.55 19.89 ...
 $ node.label : chr [1:1213] "Chemical entities" "Organic compounds" "Organoheterocyclic c
 - attr(*, "class")= chr "phylo"
 - attr(*, "order")= chr "cladewise"
```

The output of the `generate_taxonomy()` function is a list of two objects, the `phlyo` object representing the tree, and a data frame used in creating the `phylo` object. Observe the structure of these in the code chunk above.

The `phylo` object itself is a list, describing the edges of the tree, the number of internal nodes, the tip labels and internal node labels, and a set of edge lengths generated in the construction of the tree. One thing to note is that the edge lengths are artificial and are only used to speed up the creation of the tree, but do not represent any actual data corresponding to the tree. However, we retain the edge lengths as they are useful in creating visualizations.

To construct a tree for other taxonomies, one can enter in a path to a JSON file or an enter in a data frame. The JSON file must have three columns, the first the name of the label `Name`, the second its identifier `ID`, and the third the parent identifier `Parent_ID`. For the root, the parent ID should be `NA`. If the data is in the form of a data frame, it must be organized in the same way the JSON file is, with column names given by the same column names in the same order.

Next, we use the ClassyFire tool to provide classifications for the chemicals in our data sets. To simplify this process, **treecompareR** has a data pipeline that allows for programmatic acquisition of this data. It leverages the package **classyfireR** https://github.com/aberHRML/classyfireR for accessing the ClassyFire API. For chemical missing certain identifying data such as InChIKey or SMILES string data, we can use functions from the R package **ctxR** . We demonstrate how to use these below.

```
R> CASRN <- c('117964-21-3', '11006-76-1')
R> chemical_search <- ctxR::chemical_equal_batch(word_list = CASRN)
R> chemical_ids <- ctxR::get_chemical_details_batch(DTXSID = chemical_search$dtxsid)
R> data.table::setnames(x = chemical_ids, old = 'inchikey', new = 'INCHIKEY')
R> chemicals_class <- classify_datatable(chemical_ids[, .(INCHIKEY, dtxsid, casrn)])
R> str(chemicals_class)

Classes 'data.table' and 'data.frame':        2 obs. of  17 variables:
 $ INCHIKEY  : chr  NA "AAFUUKPTSPVXJH-UHFFFAOYSA-N"
 $ dtxsid    : chr  "DTXSID40880080" "DTXSID9074775"
 $ casrn     : chr  "11006-76-1" "117964-21-3"
 $ smiles    : chr  NA "BrC1=CC(Br)=C(Br)C(Br)=C1OC1=C(Br)C(Br)=C(Br)C=C1Br"
 $ inchikey  : chr  NA "InChIKey=AAFUUKPTSPVXJH-UHFFFAOYSA-N"
 $ report    : chr  NA "ClassyFire returned a classification"
 $ kingdom   : chr  NA "Organic compounds"
 $ superclass: chr  NA "Benzenoids"
 $ class     : chr  NA "Benzene and substituted derivatives"
 $ subclass  : chr  NA "Diphenylethers"
 $ level5    : chr  NA "Bromodiphenyl ethers"
 $ level6    : chr  NA NA
 $ level7    : chr  NA NA
 $ level8    : chr  NA NA
 $ level9    : chr  NA NA
 $ level10   : chr  NA NA
 $ level11   : chr  NA NA
 - attr(*, ".internal.selfref")=<externalptr>
 - attr(*, "sorted")= chr "INCHIKEY"
```

In this example, we input into the `ctxR::chemical_equal_batch()` function the list of CASRN strings. This searches and returns chemicals that match these values exactly. Next, the function `ctxR::get_chemical_details_batch()` uses the `dtxsid` column values to retrieve the chemical details for these. Finally, we input as a `data.table` with columns `INCHIKEY`, `dtxsid`, and `casrn` into the function `classify_datatable()`, which hits the ClassyFire API and attaches the retrieved information to the input data, as displayed. Note that the `CASRN` with value '117964-21-3' has both an `inchikey` and `smiles` value while the `CASRN` with value '11006-76-1' has neither piece of information. The ClassyFire data is available for the first chemical, but not the second.

We use the classified datasets `biosolids_classified` and `usgswater_classified` in several examples to follow.

If we examine the `usgswater_classified` data as follows, we get the contents of

```
R> usgswater_classified[1:5, c('INCHIKEY', 'kingdom', 'superclass', 'class', 'subclass', '
```

2. Models and software

INCHIKEY	kingdom	superclass	class	subclass
AAEVYOVXGOFMJO-UHFFFAOYSA-N	Organic compounds	Organoheterocyclic compounds	Triazines	1,3,5-triazine
AAPVQEMYVNZIOO-UHFFFAOYSA-N	Organic compounds	Organic acids and derivatives	Organic sulfuric acids and derivatives	Sulfuric acid
ADWTYURAFSWNSU-UHFFFAOYSA-N	Organic compounds	Organoheterocyclic compounds	Pyridines and derivatives	Halopyridine
AFCCDDWKHLHPDF-UHFFFAOYSA-M	Organic compounds	Organic salts	Organic metal salts	Organic alka
AFPRJLBZLPBTPZ-UHFFFAOYSA-N	Organic compounds	Benzenoids	Naphthalenes	NA

Table 1: Overview of data contained in the `usgswater_classified` classifications.

The basic Poisson regression model for count data is a special case of the GLM framework McCullagh and Nelder (1989). It describes the dependence of a count response variable y_i ($i = 1, \ldots, n$) by assuming a Poisson distribution $y_i \sim \text{Pois}(\mu_i)$. The dependence of the conditional mean $\mathsf{E}[y_i \,|\, x_i] = \mu_i$ on the regressors x_i is then specified via a log link and a linear predictor

$$\log(\mu_i) \quad = \quad x_i^\top \beta, \tag{1}$$

where the regression coefficients β are estimated by maximum likelihood (ML) using the iterative weighted least squares (IWLS) algorithm.

> Note that around the `{equation}` above there should be no spaces (avoided in the LaTeX code by `%` lines) so that "normal" spacing is used and not a new paragraph started.

R provides a very flexible implementation of the general GLM framework in the function `glm()` (Chambers and Hastie 1992) in the **stats** package. Its most important arguments are

```
glm(formula, data, subset, na.action, weights, offset,
  family = gaussian, start = NULL, control = glm.control(...),
  model = TRUE, y = TRUE, x = FALSE, ...)
```

where `formula` plus `data` is the now standard way of specifying regression relationships in R/S introduced in Chambers and Hastie (1992). The remaining arguments in the first line (`subset`, `na.action`, `weights`, and `offset`) are also standard for setting up formula-based regression models in R/S. The arguments in the second line control aspects specific to GLMs while the arguments in the last line specify which components are returned in the fitted model object (of class '`glm`' which inherits from '`lm`'). For further arguments to `glm()` (including alternative specifications of starting values) see `?glm`. For estimating a Poisson model `family = poisson` has to be specified.

> As the synopsis above is a code listing that is not meant to be executed, one can use either the dedicated `{Code}` environment or a simple `{verbatim}` environment for this. Again, spaces before and after should be avoided.
>
> Finally, there might be a reference to a `{table}` such as Table 2. Usually, these are placed at the top of the page (`[t!]`), centered (`\centering`), with a caption below the table, column headers and captions in sentence style, and if possible avoiding vertical lines.

3. Illustrations

Type	Distribution	Method	Description
GLM	Poisson	ML	Poisson regression: classical GLM, estimated by maximum likelihood (ML)
		Quasi	"Quasi-Poisson regression": same mean function, estimated by quasi-ML (QML) or equivalently generalized estimating equations (GEE), inference adjustment via estimated dispersion parameter
		Adjusted	"Adjusted Poisson regression": same mean function, estimated by QML/GEE, inference adjustment via sandwich covariances
	NB	ML	NB regression: extended GLM, estimated by ML including additional shape parameter
Zero-augmented	Poisson	ML	Zero-inflated Poisson (ZIP), hurdle Poisson
	NB	ML	Zero-inflated NB (ZINB), hurdle NB

Table 2: Overview of various count regression models. The table is usually placed at the top of the page (`[t!]`), centered (`centering`), has a caption below the table, column headers and captions are in sentence style, and if possible vertical lines should be avoided.

For a simple illustration of basic Poisson and NB count regression the `quine` data from the **MASS** package is used. This provides the number of `Days` that children were absent from school in Australia in a particular year, along with several covariates that can be employed as regressors. The data can be loaded by

```
R> data("quine", package = "MASS")
```

and a basic frequency distribution of the response variable is displayed in Figure 1.

> For code input and output, the style files provide dedicated environments. Either the "agnostic" `{CodeInput}` and `{CodeOutput}` can be used or, equivalently, the environments `{Sinput}` and `{Soutput}` as produced by `Sweave()` or **knitr** when using the `render_sweave()` hook. Please make sure that all code is properly spaced, e.g., using `y = a + b * x` and *not* `y=a+b*x`. Moreover, code input should use "the usual" command prompt in the respective software system. For R code, the prompt `"R> "` should be used with `"+ "` as the continuation prompt. Generally, comments within the code chunks should be avoided – and made in the regular LaTeX text instead. Finally, empty lines before and after code input/output should be avoided (see above).

As a first model for the `quine` data, we fit the basic Poisson regression model. (Note that JSS prefers when the second line of code is indented by two spaces.)

```
R> m_pois <- glm(Days ~ (Eth + Sex + Age + Lrn)^2, data = quine,
+    family = poisson)
```

To account for potential overdispersion we also consider a negative binomial GLM.

```
R> library("MASS")
R> m_nbin <- glm.nb(Days ~ (Eth + Sex + Age + Lrn)^2, data = quine)
```

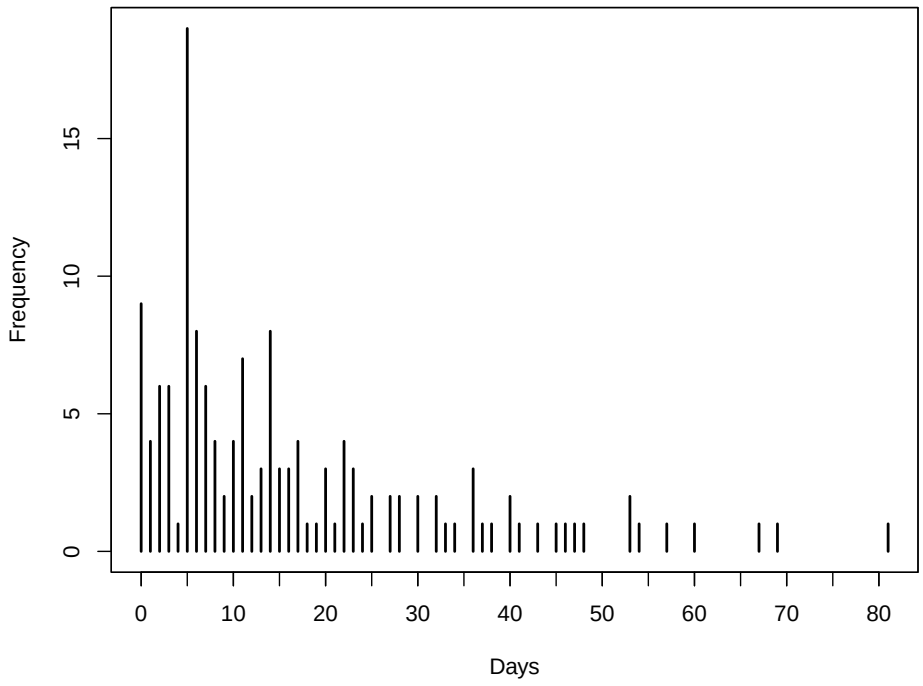

Figure 1: Frequency distribution for number of days absent from school.

In a comparison with the BIC the latter model is clearly preferred.

```
R> BIC(m_pois, m_nbin)

       df      BIC
m_pois 18 2046.851
m_nbin 19 1157.235
```

Hence, the full summary of that model is shown below.

```
R> summary(m_nbin)

Call:
glm.nb(formula = Days ~ (Eth + Sex + Age + Lrn)^2, data = quine,
    init.theta = 1.60364105, link = log)

Coefficients: (1 not defined because of singularities)
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.00155    0.33709   8.904  < 2e-16 ***
EthN        -0.24591    0.39135  -0.628  0.52977
SexM        -0.77181    0.38021  -2.030  0.04236 *
AgeF1       -0.02546    0.41615  -0.061  0.95121
AgeF2       -0.54884    0.54393  -1.009  0.31296
AgeF3       -0.25735    0.40558  -0.635  0.52574
```

```
LrnSL        0.38919    0.48421   0.804  0.42153
EthN:SexM    0.36240    0.29430   1.231  0.21818
EthN:AgeF1  -0.70000    0.43646  -1.604  0.10876
EthN:AgeF2  -1.23283    0.42962  -2.870  0.00411 **
EthN:AgeF3   0.04721    0.44883   0.105  0.91622
EthN:LrnSL   0.06847    0.34040   0.201  0.84059
SexM:AgeF1   0.02257    0.47360   0.048  0.96198
SexM:AgeF2   1.55330    0.51325   3.026  0.00247 **
SexM:AgeF3   1.25227    0.45539   2.750  0.00596 **
SexM:LrnSL   0.07187    0.40805   0.176  0.86019
AgeF1:LrnSL -0.43101    0.47948  -0.899  0.36870
AgeF2:LrnSL  0.52074    0.48567   1.072  0.28363
AgeF3:LrnSL       NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(1.6036) family taken to be 1)

    Null deviance: 235.23  on 145  degrees of freedom
Residual deviance: 167.53  on 128  degrees of freedom
AIC: 1100.5

Number of Fisher Scoring iterations: 1


              Theta:  1.604
          Std. Err.:  0.214

 2 x log-likelihood:  -1062.546
```

4. Summary and discussion

- As usual ...

Computational details

If necessary or useful, information about certain computational details such as version numbers, operating systems, or compilers could be included in an unnumbered section. Also, auxiliary packages (say, for visualizations, maps, tables, ...) that are not cited in the main text can be credited here.

The results in this paper were obtained using R .4 with the **MASS** 7.3.61 package. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at https://CRAN.R-project.org/.

Acknowledgments

All acknowledgments (note the AE spelling) should be collected in this unnumbered section before the references. It may contain the usual information about funding and feedback from colleagues/reviewers/etc. Furthermore, information such as relative contributions of the authors may be added here (if any).

References

Chambers JM, Hastie TJ (eds.) (1992). *Statistical Models in S*. Chapman & Hall, London.

Djoumbou Feunang Y, Eisner R, Knox C, Chepelev L, Hastings J, Owen G, Fahy E, Steinbeck C, Subramanian S, Bolton E, *et al.* (2016). "ClassyFire: automated chemical classification with a comprehensive, computable taxonomy." *Journal of cheminformatics*, **8**, 1–20.

Gu Z, Eils R, Schlesner M (2016). "Complex heatmaps reveal patterns and correlations in multidimensional genomic data." *Bioinformatics*. doi:10.1093/bioinformatics/btw313.

McCullagh P, Nelder JA (1989). *Generalized Linear Models*. 2nd edition. Chapman & Hall, London. doi:10.1007/978-1-4899-3242-6.

Paradis E, Schliep K (2019). "ape 5.0: an environment for modern phylogenetics and evolutionary analyses in R." *Bioinformatics*, **35**, 526–528. doi:10.1093/bioinformatics/bty633.

Revell LJ (2024). "phytools 2.0: an updated R ecosystem for phylogenetic comparative methods (and other things)." *PeerJ*, **12**, e16505. doi:10.7717/peerj.16505.

Schliep K (2011). "phangorn: phylogenetic analysis in R." *Bioinformatics*, **27**(4), 592–593. doi:10.1093/bioinformatics/btq706.

Schliep K, Potts AJ, Morrison DA, Grimm GW (2017). "Intertwining phylogenetic trees and networks." *Methods in Ecology and Evolution*, **8**(10), 1212–1220.

Wang LG, Lam TTY, Xu S, Dai Z, Zhou L, Feng T, Guo P, Dunn CW, Jones BR, Bradley T, Zhu H, Guan Y, Jiang Y, Yu G (2020). "treeio: an R package for phylogenetic tree input and output with richly annotated and associated data." *Molecular Biology and Evolution*, **37**, 599–603. doi:10.1093/molbev/msz240.

Wickham H (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. ISBN 978-3-319-24277-4. URL https://ggplot2.tidyverse.org.

Williams AJ, Grulke CM, Edwards J, McEachran AD, Mansouri K, Baker NC, Patlewicz G, Shah I, Wambaugh JF, Judson RS, *et al.* (2017). "The CompTox Chemistry Dashboard: a community data resource for environmental chemistry." *Journal of cheminformatics*, **9**, 1–27.

Yu G, Smith DK, Zhu H, Guan Y, Lam TTY (2017). "ggtree: an R package for visualization and annotation of phylogenetic trees with their covariates and other associated data." *Methods in Ecology and Evolution*, **8**(1), 28–36.

A. More technical details

Appendices can be included after the bibliography (with a page break). Each section within the appendix should have a proper section title (rather than just *Appendix*).

For more technical style details, please check out JSS's style FAQ at https://www.jstatsoft.org/pages/view/style#frequently-asked-questions which includes the following topics:

- Title vs. sentence case.
- Graphics formatting.
- Naming conventions.
- Turning JSS manuscripts into R package vignettes.
- Trouble shooting.
- Many other potentially helpful details...

B. Using BibTeX

References need to be provided in a BibTeX file (`.bib`). All references should be made with `\cite`, `\citet`, `\citep`, `\citealp` etc. (and never hard-coded). This commands yield different formats of author-year citations and allow to include additional details (e.g., pages, chapters, ...) in brackets. In case you are not familiar with these commands see the JSS style FAQ for details.

Cleaning up BibTeX files is a somewhat tedious task – especially when acquiring the entries automatically from mixed online sources. However, it is important that informations are complete and presented in a consistent style to avoid confusions. JSS requires the following format.

- JSS-specific markup (`\proglang`, `\pkg`, `\code`) should be used in the references.
- Titles should be in title case.
- Journal titles should not be abbreviated and in title case.
- DOIs should be included where available.
- Software should be properly cited as well. For R packages `citation("pkgname")` typically provides a good starting point.

Affiliation:

Paul Kruse
Center for Computational Toxicology and Exposure
United States Environmental Protection Agency
109 TW Alexander Dr, Durham, NC 27709, United States of America
E-mail: kruse.paul@epa.gov

http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd